```
#monalika_kubde-23070521094

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Bidirectional
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

sentences = [
    "book a flight from mumbai to delhi",
    "i want a flight from delhi to bangalore",
    "show flights from pune to mumbai",
    "find a flight from chennai to delhi",
    "book a ticket from nagpur to goa",
    "i need a flight from hyderabad to pune",
    "find flights from bangalore to chennai",
    "book flight from kolkata to mumbai",
    "show me flights from delhi to goa",
    "i want to travel from pune to delhi",
    "book a flight from jaipur to mumbai",
    "find a flight from surat to delhi",
    "show flights from goa to pune",
    "i need a ticket from mumbai to chennai",
    "book a flight from delhi to hyderabad",
    "find flights from nagpur to bangalore",
    "i want a ticket from pune to kolkata",
    "book flight from chennai to mumbai",
    "show me flights from goa to delhi",
    "find a flight from hyderabad to bangalore"
]

labels = [
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","O","B-fromloc.city","O","B-toloc.city"],
    ["O","O","O","O","B-fromloc.city","O","B-toloc.city"]
]

print("\n====================================")
print("TEXT PREPROCESSING")
print("====================================")

# Create tokenizer
tokenizer = Tokenizer(
    lower=True,
    oov_token="<OOV>"
)

tokenizer.fit_on_texts(sentences)

# Word dictionary
word_index = tokenizer.word_index

print("\nWord Index:")
```

```
print(word_index)

# Convert sentences into numbers
X = tokenizer.texts_to_sequences(sentences)

# Maximum sentence length
MAX_LEN = max(len(x) for x in X)

print("\nMaximum sentence length:", MAX_LEN)

# Padding
X = pad_sequences(
    X,
    maxlen=MAX_LEN,
    padding="post"
)

print("\nPreprocessed Sentences:")
print(X)

print("\n====================================")
print("LABEL ENCODING")
print("====================================")

# Find unique labels
unique_labels = sorted(
    set(label for sequence in labels for label in sequence)
)

# Create label-to-ID mapping
label2id = {
    label: i + 1
    for i, label in enumerate(unique_labels)
}

# 0 is reserved for padding
label2id["PAD"] = 0

# Reverse mapping
id2label = {
    value: key
    for key, value in label2id.items()
}

print("\nLabel Mapping:")
print(label2id)


# Convert labels into numbers
Y = []

for sequence in labels:

    encoded = [
        label2id[label]
        for label in sequence
    ]

    # Padding
    while len(encoded) < MAX_LEN:
        encoded.append(label2id["PAD"])

    Y.append(encoded)

Y = np.array(Y)

print("\nEncoded Labels:")
print(Y)

X_train, X_test, Y_train, Y_test = train_test_split(
    X,
    Y,
    test_size=0.20,
    random_state=42
)

print("\n====================================")
print("DATASET SPLIT")
```

```
print("====================================")

print("Training samples:", len(X_train))
print("Testing samples :", len(X_test))

print("\n====================================")
print("BUILDING LSTM MODEL")
print("====================================")

VOCAB_SIZE = len(word_index) + 1
EMBEDDING_DIM = 64
LSTM_UNITS = 64
NUM_CLASSES = len(label2id)

model = Sequential()

# Embedding layer
model.add(
    Embedding(
        input_dim=VOCAB_SIZE,
        output_dim=EMBEDDING_DIM
    )
)

# Bidirectional LSTM
model.add(
    Bidirectional(
        LSTM(
            LSTM_UNITS,
            return_sequences=True
        )
    )
)

# Output layer
model.add(
    Dense(
        NUM_CLASSES,
        activation="softmax"
    )
)

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

print("\nModel Summary:")
model.summary()

print("\n====================================")
print("TRAINING MODEL")
print("====================================")

history = model.fit(
    X_train,
    Y_train,
    epochs=30,
    batch_size=2,
    validation_split=0.2,
    verbose=1
)

print("\n====================================")
print("MODEL EVALUATION")
print("====================================")

loss, accuracy = model.evaluate(
    X_test,
    Y_test,
    verbose=0
)

print("\nTest Loss     :", loss)
print("Test Accuracy :", accuracy)
```

```
Y_pred = model.predict(
    X_test,
    verbose=0
)

# Convert probabilities to class IDs
Y_pred = np.argmax(
    Y_pred,
    axis=-1
)

true_labels = []
pred_labels = []

for true_sequence, pred_sequence in zip(
    Y_test,
    Y_pred
):

    for true_id, pred_id in zip(
        true_sequence,
        pred_sequence
    ):

        # Ignore padding
        if true_id != label2id["PAD"]:

            true_labels.append(
                id2label[true_id]
            )

            pred_labels.append(
                id2label.get(
                    pred_id,
                    "O"
                )
            )


token_accuracy = accuracy_score(
    true_labels,
    pred_labels
)

print("\nToken Level Accuracy:",
      token_accuracy)

print("\n====================================")
print("CLASSIFICATION REPORT")
print("====================================")

print(
    classification_report(
        true_labels,
        pred_labels,
        zero_division=0
    )
)

plt.figure(figsize=(9, 5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy",
    linewidth=2,
    marker="o"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy",
    linewidth=2,
    marker="o"
)

plt.title(
    "LSTM Slot Filling - Training and Validation Accuracy"
)
```

```
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

plt.figure(figsize=(9, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss",
    linewidth=2,
    marker="o"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss",
    linewidth=2,
    marker="o"
)

plt.title(
    "LSTM Slot Filling - Training and Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

def predict_slots(sentence):

    print("\n====================================")
    print("SLOT PREDICTION")
    print("====================================")

    print("Sentence:", sentence)

    # Convert sentence to numbers
    sequence = tokenizer.texts_to_sequences(
        [sentence]
    )

    # Padding
    padded = pad_sequences(
        sequence,
        maxlen=MAX_LEN,
        padding="post"
    )

    # Predict
    prediction = model.predict(
        padded,
        verbose=0
    )

    # Select highest probability
    prediction = np.argmax(
        prediction,
        axis=-1
    )[0]

    # Split sentence into words
    words = sentence.lower().split()

    print("\nWord                Slot")
    print("--------------------------------")
```

```
    for i, word in enumerate(words):

        if i < MAX_LEN:

            slot = id2label.get(
                prediction[i],
                "O"
            )

            print(
                f"{word:<20} {slot}"
            )

predict_slots(
    "book a flight from mumbai to delhi"
)

predict_slots(
    "find a flight from pune to goa"
)

predict_slots(
    "book a flight from nagpur to chennai"
)
```

```
======================================
TEXT PREPROCESSING
======================================

Word Index:
{'<OOV>': 1, 'to': 2, 'from': 3, 'a': 4, 'flight': 5, 'delhi': 6, 'book': 7, 'mumbai': 8, 'flights': 9, 'i': 10, 'pune': 11, 't

Maximum sentence length: 8

Preprocessed Sentences:
[[ 7  4  5  3  8  2  6  0]
 [10 17  4  5  3  6  2 13]
 [14  9  3 11  2  8  0  0]
 [12  4  5  3 15  2  6  0]
 [ 7  4 18  3 20  2 16  0]
 [10 21  4  5  3 19  2 11]
 [12  9  3 13  2 15  0  0]
 [ 7  5  3 22  2  8  0  0]
 [14 23  9  3  6  2 16  0]
 [10 17  2 24  3 11  2  6]
 [ 7  4  5  3 25  2  8  0]
 [12  4  5  3 26  2  6  0]
 [14  9  3 16  2 11  0  0]
 [10 21  4 18  3  8  2 15]
 [ 7  4  5  3  6  2 19  0]
 [12  9  3 20  2 13  0  0]
 [10 17  4 18  3 11  2 22]
 [ 7  5  3 15  2  8  0  0]
 [14 23  9  3 16  2  6  0]
 [12  4  5  3 19  2 13  0]]

======================================
LABEL ENCODING
======================================

Label Mapping:
{'B-fromloc.city': 1, 'B-toloc.city': 2, 'O': 3, 'PAD': 0}

Encoded Labels:
[[3 3 3 3 1 3 2 0]
 [3 3 3 3 1 3 2 0]
 [3 3 3 3 1 3 2 0]
 [3 3 3 3 1 3 2 0]
 [3 3 3 3 1 3 2 0]
 [3 3 3 3 1 3 2 0]
 [3 3 3 3 1 3 2 0]
 [3 3 3 3 1 3 2 0]
 [3 3 3 3 3 1 3 2]
 [3 3 3 3 1 3 2 0]
 [3 3 3 3 1 3 2 0]
 [3 3 3 3 1 3 2 0]
 [3 3 3 3 1 3 2 0]
 [3 3 3 3 1 3 2 0]
 [3 3 3 3 1 3 2 0]
 [3 3 3 3 1 3 2 0]
 [3 3 3 3 1 3 2 0]
 [3 3 3 3 3 1 3 2]
 [3 3 3 3 1 3 2 0]]

======================================
DATASET SPLIT
======================================
Training samples: 16
Testing samples : 4

======================================
BUILDING LSTM MODEL
======================================

Model Summary:
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
bidirectional (Bidirectional)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

```
 Non-trainable params: 0 (0.00 B)

====================================
TRAINING MODEL
====================================
Epoch 1/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 9s 124ms/step - accuracy: 0.5000 - loss: 1.3655 - val_accuracy: 0.6250 - val_loss: 1.3328
Epoch 2/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.6458 - loss: 1.2989 - val_accuracy: 0.6250 - val_loss: 1.2542
Epoch 3/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.6458 - loss: 1.1926 - val_accuracy: 0.6250 - val_loss: 1.1210
Epoch 4/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.6458 - loss: 1.0240 - val_accuracy: 0.6250 - val_loss: 0.9179
Epoch 5/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.6458 - loss: 0.8277 - val_accuracy: 0.6250 - val_loss: 0.7981
Epoch 6/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.6458 - loss: 0.7619 - val_accuracy: 0.6250 - val_loss: 0.7699
Epoch 7/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.6458 - loss: 0.7221 - val_accuracy: 0.6250 - val_loss: 0.7141
Epoch 8/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.6875 - loss: 0.6760 - val_accuracy: 0.7188 - val_loss: 0.6765
Epoch 9/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 0.7604 - loss: 0.6449 - val_accuracy: 0.8125 - val_loss: 0.6441
Epoch 10/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.7708 - loss: 0.6095 - val_accuracy: 0.8125 - val_loss: 0.5952
Epoch 11/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 0.7812 - loss: 0.5627 - val_accuracy: 0.8438 - val_loss: 0.5452
Epoch 12/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step - accuracy: 0.8333 - loss: 0.5130 - val_accuracy: 0.8438 - val_loss: 0.4942
Epoch 13/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.8438 - loss: 0.4630 - val_accuracy: 0.8438 - val_loss: 0.4445
Epoch 14/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.8542 - loss: 0.4119 - val_accuracy: 0.8438 - val_loss: 0.3998
Epoch 15/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.8542 - loss: 0.3684 - val_accuracy: 0.8438 - val_loss: 0.3640
Epoch 16/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.8750 - loss: 0.3297 - val_accuracy: 0.8438 - val_loss: 0.3486
Epoch 17/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8854 - loss: 0.2948 - val_accuracy: 0.8438 - val_loss: 0.3053
Epoch 18/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.8958 - loss: 0.2676 - val_accuracy: 0.8438 - val_loss: 0.3160
Epoch 19/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.8854 - loss: 0.2637 - val_accuracy: 0.8750 - val_loss: 0.2443
Epoch 20/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9375 - loss: 0.2378 - val_accuracy: 0.8438 - val_loss: 0.2701
Epoch 21/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.8958 - loss: 0.2185 - val_accuracy: 0.8750 - val_loss: 0.2225
Epoch 22/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9375 - loss: 0.1986 - val_accuracy: 0.9062 - val_loss: 0.2087
Epoch 23/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9479 - loss: 0.1854 - val_accuracy: 0.9375 - val_loss: 0.2110
Epoch 24/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9271 - loss: 0.1686 - val_accuracy: 0.9062 - val_loss: 0.1914
Epoch 25/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9688 - loss: 0.1534 - val_accuracy: 0.9375 - val_loss: 0.1745
Epoch 26/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9792 - loss: 0.1452 - val_accuracy: 0.9375 - val_loss: 0.1763
Epoch 27/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9583 - loss: 0.1318 - val_accuracy: 0.9062 - val_loss: 0.1594
Epoch 28/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.9688 - loss: 0.1284 - val_accuracy: 1.0000 - val_loss: 0.1414
Epoch 29/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9792 - loss: 0.1146 - val_accuracy: 0.9375 - val_loss: 0.1525
Epoch 30/30
6/6 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.9896 - loss: 0.1082 - val_accuracy: 1.0000 - val_loss: 0.1189

====================================
MODEL EVALUATION
====================================

Test Loss     : 0.11705749481916428
Test Accuracy : 1.0
```